Learner Guide

Installing NVIDIA Isaac Lab on Windows 11

Local installation using the Isaac Sim pip package and Isaac Lab from source

Item	Detail
Target OS	Windows 11 (x64)
Isaac Sim version	5.1.0 (installed via pip)
Python version	3.11
PyTorch	2.7.0 + torchvision 0.22.0 (CUDA 12.8 build, cu128)
Estimated time	60–120 minutes (mostly downloads and first-run extension caching)
Source	Isaac Lab documentation, “Installation using Isaac Sim Pip Package” (main branch), retrieved 12 October 2026

Official page: isaac-sim.github.io/IsaacLab/main/source/setup/installation/pip_installation.html

Contents

Contents 2
1. About This Guide 3
Learning outcomes 3
Conventions used in this guide 3
2. System Requirements 3
3. Before You Begin: Prepare Windows 4
Prep 1. Check your GPU and driver 4
Prep 2. Enable long path support 4
Prep 3. Install Git 4
Prep 4. Install a Python environment manager 4
4. Installation Steps 5
Step 1. Create a working folder 5
Step 2. Create and activate a Python 3.11 environment 5
Step 3. Upgrade pip 5
Step 4. Install Isaac Sim 5.1.0 6
Step 5. Install the matching PyTorch build 6
Step 6. Verify Isaac Sim and accept the EULA 6
Step 7. Clone the Isaac Lab repository 7
Step 8. Install Isaac Lab 7
5. Verify the Installation 8
Step 9. Run the empty-scene tutorial 8
Step 10. Train your first robot (optional but recommended) 8
6. Troubleshooting 9
7. Quick Reference — Full Command Sequence 9
8. Intentional Omissions 10

(If the table of contents is empty, right-click it in Word and choose Update Field.)

1. About This Guide

NVIDIA Isaac Lab is an open-source framework for robot learning (reinforcement learning, imitation learning and motion planning) built on top of NVIDIA Isaac Sim. This guide walks you through installing Isaac Lab on your own Windows 11 PC so that you can run the simulator, the tutorials and the reinforcement-learning training examples used in class.

The installation has two parts:

- **Isaac Sim** is installed as a Python package with pip into a dedicated Python 3.11 environment.
- **Isaac Lab** is cloned from GitHub and installed from source into the same environment using the isaaclab.bat helper script.

Learning outcomes

By the end of this guide you will be able to:

- Prepare a Windows 11 PC that meets Isaac Sim's hardware and driver requirements.
- Create and activate an isolated Python 3.11 environment with Conda or venv.
- Install Isaac Sim 5.1.0 and the matching CUDA build of PyTorch with pip.
- Clone and install Isaac Lab with its reinforcement-learning frameworks.
- Verify the installation and launch a headless training run.

Conventions used in this guide

- All commands are for the **Windows Command Prompt** (cmd.exe) or the **Anaconda Prompt** unless a box is labelled PowerShell.
- Lines starting with `::` inside a command box are comments — you do not need to type them.
- Text in `monospace` is something you type or see on screen exactly as shown.
- A **blue box** is a note, a **red box** is a warning, and a **yellow box** is a checkpoint.

2. System Requirements

Check your PC against the table below before you start. Isaac Sim is a GPU-heavy RTX application; under-specified machines will either fail to launch or run extremely slowly.

Component	Requirement
Operating system	Windows 11 (x64)
GPU	NVIDIA RTX-class GPU with 16 GB VRAM or more (rendering workflows may need more)
System RAM	32 GB or more
NVIDIA driver	Latest production-branch driver; 580.88 is the recommended Windows version
Python	3.11 (must match Isaac Sim 5.x — a different version will cause errors)
Disk space	Plan for several tens of GB free on an SSD (Isaac Sim packages, extension cache, PyTorch and Isaac Lab)
Network	Stable internet connection; the first launch downloads extensions and can take over 10 minutes

> **Note on disk space**
> The official page does not state a disk-space figure. The estimate above is guidance from this guide, not an NVIDIA requirement.

3. Before You Begin: Prepare Windows

Prep 1. Check your GPU and driver

Open Command Prompt (press Start, type cmd, press Enter) and run:

Command Prompt

```
nvidia-smi
```

Expected result: A table showing your GPU name, its memory (e.g. 16384MiB) and a Driver Version. If the driver version is older than 580.88, download the latest production driver from nvidia.com/drivers, install it and restart.

Prep 2. Enable long path support

Isaac Sim and Isaac Lab contain deeply nested folders. The official documentation warns that on Windows it may be necessary to enable long path support to avoid installation errors. Open PowerShell as Administrator (right-click Start → Terminal (Admin)) and run:

PowerShell (Administrator)

```
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem" `
  -Name "LongPathsEnabled" -Value 1 -PropertyType DWORD -Force
```

Restart your PC afterwards. Reference: Microsoft's "Maximum Path Length Limitation" article, linked from the Isaac Lab page.

> **Tip**
> Keep your working folder short and close to the drive root, for example C:\isaac. Avoid paths with spaces or OneDrive-synced folders such as Documents or Desktop.

Prep 3. Install Git

Download and install Git for Windows from git-scm.com (default options are fine). Confirm it works:

Command Prompt

```
git --version
```

Prep 4. Install a Python environment manager

You need either Miniconda (recommended by the Isaac Lab docs because it is lightweight) or a standalone Python 3.11 installation for venv.

- **Option A — Miniconda (recommended):** install from docs.conda.io/en/latest/miniconda.html. Afterwards use the "Anaconda Prompt (miniconda3)" from the Start menu for all remaining steps.
- **Option B — venv:** install Python 3.11 (64-bit) from python.org and tick "Add python.exe to PATH".

4. Installation Steps

Step 1. Create a working folder

Command Prompt / Anaconda Prompt

```
mkdir C:\isaac
cd C:\isaac
```

Step 2. Create and activate a Python 3.11 environment

Choose ONE option. The environment is named env_isaaclab throughout this guide.

Option A — Conda

Anaconda Prompt

```
conda create -n env_isaaclab python=3.11
conda activate env_isaaclab
```

Option B — venv

Command Prompt

```
:: create a virtual environment named env_isaaclab with python3.11
python3.11 -m venv env_isaaclab
:: activate the virtual environment
env_isaaclab\Scripts\activate
```

> **If python3.11 is not recognised**
>
> On many Windows installations the Python launcher is used instead. Try: py -3.11 -m venv env_isaaclab

Expected result: Your prompt now begins with (env_isaaclab). Every remaining step must be run in this activated environment.

> **Python version must match**
>
> Isaac Sim 5.x is built against Python 3.11. Using a different Python version will result in errors when running Isaac Lab. Check with: python --version

Step 3. Upgrade pip

On Windows, upgrade pip through the python module form of the command:

Activated environment

```
python -m pip install --upgrade pip
```

Step 4. Install Isaac Sim 5.1.0

Isaac Sim is distributed through NVIDIA's own package index. This is a large download.

Activated environment

```
pip install "isaacsim[all,extscache]==5.1.0" --extra-index-url
https://pypi.nvidia.com
```

Step 5. Install the matching PyTorch build

Install the CUDA 12.8 build of PyTorch specified for Windows (x86_64):

Activated environment

```
pip install -U torch==2.7.0 torchvision==0.22.0 --index-url
https://download.pytorch.org/whl/cu128
```

Optional quick check that PyTorch can see your GPU:

Activated environment

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

Expected result: 2.7.0+cu128 True

Step 6. Verify Isaac Sim and accept the EULA

Launch Isaac Sim once from the activated environment:

Activated environment

```
:: note: you can pass the argument "--help" to see all arguments possible.
isaacsim
```

On the first run you will be asked to accept the NVIDIA Omniverse License Agreement. Type Yes and press Enter:

Terminal prompt (what you will see)

```
By installing or using Isaac Sim, I agree to the terms of NVIDIA OMNIVERSE LICENSE AGREEMENT
(EULA)
in
https://docs.isaacsim.omniverse.nvidia.com/latest/common/NVIDIA_Omniverse_License_Agreement.html

Do you accept the EULA? (Yes/No): Yes
```

> **Be patient on the first run**
>
> The first launch downloads and caches extensions and can take upwards of 10 minutes. This is required on the first run of each experience file; later launches are much faster.
>
> To launch a specific experience file instead: isaacsim isaacsim.exp.full.kit

Expected result: The Isaac Sim application window opens. Close it once it has loaded.

> **Checkpoint 1**
> Isaac Sim launches from your env_isaaclab environment. Do not continue until this works — see Section 6 if it does not.

Step 7. Clone the Isaac Lab repository

Return to your working folder and clone the main branch. Use HTTPS unless you have SSH keys set up with GitHub.

Activated environment

```
cd C:\isaac
:: Option 1: HTTPS
git clone https://github.com/isaac-sim/IsaacLab.git

:: Option 2: SSH
git clone git@github.com:isaac-sim/IsaacLab.git

cd IsaacLab
```

> **Forking (optional)**
> Forking the repository is recommended if you plan to contribute, but it is not required. If you fork, replace isaac-sim in the URL with your GitHub username.

Check that the Windows helper script is available:

Activated environment, inside C:\isaac\IsaacLab

```
isaaclab.bat --help
```

You should see the usage summary. The options you will use most are:

Option	Purpose
`-i, --install [LIB]`	Install Isaac Lab extensions and learning frameworks (default: all)
`-p, --python`	Run the Python of the active virtual environment (or bundled Isaac Sim Python)
`-s, --sim`	Run the Isaac Sim executable (isaac-sim.bat)
`-n, --new`	Create a new external project or internal task from a template
`-v, --vscode`	Generate VS Code settings from template
`-c, --conda [NAME]`	Create a conda environment for Isaac Lab (default env_isaaclab)

Step 8. Install Isaac Lab

From inside the IsaacLab folder, install all Isaac Lab extensions together with all supported learning frameworks:

Activated environment, inside C:\isaac\IsaacLab

```
isaaclab.bat --install :: or "isaaclab.bat -i"
```

The script iterates over the extensions in the source folder and installs each one with pip in editable mode. To install only a single framework instead, pass its name, for example:

Activated environment

```
isaaclab.bat --install rl_games :: or "isaaclab.bat -i rl_games"
```

Valid values are `all`, `rl_games`, `rsl_rl`, `sb3`, `skrl`, `robomimic` and `none`.

> **Windows note on robomimic**
> The official docs state that robomimic is not available on Windows. Its Linux-only build dependencies (cmake, build-essential via apt) are therefore not needed on Windows. The RSL-RL training examples in Section 5 do not depend on it.

5. Verify the Installation

Step 9. Run the empty-scene tutorial

Use either option below from the IsaacLab folder:

Activated environment, inside C:\isaac\IsaacLab

```
:: Option 1: Using the isaaclab.bat executable
:: note: this works for both the bundled python and the virtual environment
isaaclab.bat -p scripts\tutorials\00_sim\create_empty.py

:: Option 2: Using python in your virtual environment
python scripts\tutorials\00_sim\create_empty.py
```

Expected result: A simulator window opens with a black viewport. This is correct — the scene is intentionally empty.

To stop it, click in the Command Prompt window and press **Ctrl+Break** (on laptops without a Break key, try **Ctrl+Fn+B**).

> **Checkpoint 2**
> The empty scene opens without errors. Your Isaac Lab installation is working.

Step 10. Train your first robot (optional but recommended)

The --headless flag runs training without rendering the viewport, which is much faster.

Example A — Ant locomotion

Activated environment, inside C:\isaac\IsaacLab

```
isaaclab.bat -p scripts/reinforcement_learning/rsl_rl/train.py --task=Isaac-Ant-v0 --headless
```

Example B — ANYmal-C quadruped on rough terrain

Activated environment, inside C:\isaac\IsaacLab

```
isaaclab.bat -p scripts/reinforcement_learning/rsl_rl/train.py --task=Isaac-
Velocity-Rough-Anymal-C-v0 --headless
```

Expected result: Training iterations and reward statistics are printed in the terminal. Logs and checkpoints are written under the logs folder in IsaacLab.

6. Troubleshooting

Symptom	Likely cause and fix
ModuleNotFoundError: No module named 'isaacsim'	The environment is not activated, or you opened a new terminal. Run conda activate env_isaaclab (or env_isaaclab\Scripts\activate) and try again. (The docs also mention sourcing _isaac_sim/setup_conda_env.sh — that script applies to Linux.)
Install fails with path-too-long or file-not-found errors	Long path support is not enabled. Repeat Step 0.2, restart, and use a short working folder such as C:\isaac.
Errors about Python version / incompatible wheels	Your environment is not Python 3.11. Delete it and recreate it with Python 3.11 (Step 2).
isaacsim appears to hang on first launch	Normal: extensions are being downloaded and cached. Allow 10+ minutes on the first run.
torch.cuda.is_available() returns False	A CPU-only PyTorch was installed, or the driver is outdated. Re-run Step 5 exactly and update the NVIDIA driver.
'isaaclab.bat' is not recognized	You are not inside the IsaacLab folder. Run cd C:\isaac\IsaacLab.
Simulator window crashes or is very slow	GPU below spec (less than 16 GB VRAM) or outdated driver. Update the driver; use --headless for training.

Further help: Isaac Sim documentation and NVIDIA developer forums (linked from the Isaac Lab page), and the Isaac Lab GitHub Discussions.

7. Quick Reference — Full Command Sequence

For learners who have already completed Section 3 (driver, long paths, Git, Miniconda), the full Conda-based sequence is:

Anaconda Prompt

```
mkdir C:\isaac && cd C:\isaac
conda create -n env_isaaclab python=3.11
conda activate env_isaaclab
python -m pip install --upgrade pip
pip install "isaacsim[all,extscache]==5.1.0" --extra-index-url
https://pypi.nvidia.com
pip install -U torch==2.7.0 torchvision==0.22.0 --index-url
https://download.pytorch.org/whl/cu128
isaacsim
:: type Yes to accept the EULA, wait for the window, then close it
```

```
git clone https://github.com/isaac-sim/IsaacLab.git
cd IsaacLab
isaaclab.bat --install
isaaclab.bat -p scripts\tutorials\00_sim\create_empty.py
isaaclab.bat -p scripts/reinforcement_learning/rsl_rl/train.py --task=Isaac-Ant-v0 --headless
```

Each new session: open Anaconda Prompt, then run `conda activate env_isaaclab` and `cd C:\isaac\IsaacLab` before using Isaac Lab.

8. Intentional Omissions

The following content from the official page has been deliberately left out of this Windows learner guide:

- **Linux (x86_64) and Linux (aarch64 / DGX Spark) commands** — including ./isaaclab.sh, apt dependencies and the aarch64 libgomp LD_PRELOAD workaround.
- **uv environment option** — marked experimental in the docs; Conda and venv are the supported paths here.
- **Isaac Sim 4.x / Python 3.10 variant** — this guide targets Isaac Sim 5.1.0 only.
- **Visual Studio Code setup** — the docs recommend the binaries installation instead of pip if you intend to set up VS Code integration.
- **Asset caching, Docker and cloud deployment** — covered on separate Isaac Lab pages.

> **Additions not on the official page**
>
> The following were added for learner convenience and are not part of the NVIDIA page: the nvidia-smi and torch.cuda checks, the PowerShell command for enabling long paths, the C:\isaac folder suggestion, the py -3.11 fallback, the disk-space estimate, and the troubleshooting table entries beyond the ModuleNotFoundError and long-path items.

> **Version drift**
>
> Isaac Lab's main-branch docs change frequently. On 12 October 2026 the pip page specified Isaac Sim 5.1.0, while the docs header badge referenced a newer Isaac Sim release. If you are told to use a different Isaac Sim version, check the official page and keep the Python and PyTorch versions consistent with it.